This function gives the command to the switch to enter the flow table entry. Now, to take appropriate action at the switch, we need to create the entry for the packet that we want to route. Let's say we want to route a packet coming to input port = *2*. So, we need to create a matching object for that, which is given by *ofp_match*. To create the entry in the switch, write the following code:

```
fm = of.ofp_flow_mod()
        fm.match_in_port = 2
        fm.actions.append(of.ofp_action_output(port = 4))
```

Thus, as and when a packet comes at the port = *2*, it will be redirected to port = *4*. We can definitely add more parameters as per the requirement, like *idle_timeout*, *hard_timeout*, *actions*, *priority*, *buffer_id*, *match*, *in_port*. But we have not included all of them -- just to avoid complexity.

## An SDN based load balancer using round robin scheduling

As writing a load balancer could be a tedious task, we can use a template of an existing controller which forwards a request to the available servers randomly. The template can be found at *https://github.com/noxrepo/pox/blob/carp/pox/misc/ip_loadbalancer.py*

The main role of the controller is to select the server to which the incoming requests are to be forwarded. This part is coded in the function of the template called *pick_server*. It is found to be at Line No. 190 in the standard template. Thus, we can modify that procedure to customise the default policy.

The code below refers to Line No. 190 of the file *ip_loadbalancer.py:*

```
def _pick_server (self, key, inport):
"""
Pick a server for a (hopefully) new connection
"""
return random.choice(self.live_servers.keys())
```

The above lines show the random selection of the servers for forwarding. Now we are going to change this method to remove the randomness and add the round robin algorithm to the load balancer. First, we will add a new variable called *selected_server* to class *iplb* (which is the class containing the above method, i.e., *pick_server()* ). The *selected_server* will be an instance variable defaulting to 0, which will keep track of the state of the server allocated to fulfill the immediate previous request.

So, a modified class definition of *iplb* will be as shown below:

```
class iplb:
        def __init__(self,...):
...
...
self.selected_server = 0
```

Here, *__init__* is the constructor and the *self* inside the method refers to the instance of the object. So is the case with anything defined with *self*. *Prefixed* can be used as an instance variable (similar to other object-oriented languages such as Java). So now, to change the *pick_server* method, run the following commands:

```
 def _pick_server (self, key, inport):
 """
```

Pick a server for a (hopefully) new round robin based connection.

```
"""
all_keys=self.live_servers.keys()
 if self.selected_server==len(self.live_servers):
self.selected_server=0
redirected_s=all_keys[self.selected_server]
self.selected_server+=1
return redirected_s
```

The above code first gets all the keys from the *self.live_servers* (which is a Python dictionary, i.e., a Hash Map). Then, in the next line, it makes sure that the *selected_server* isn't out of range to the total number of servers. Finally, it gets the key of the selected server and then increments the variable for the next call. END

## References


[1] *http://www.blackstratus.com/blog/traditional-software-defined-networking/*
[2] *https://globalconfig.net/software-defined-networking-vs-traditional/*
[3] *https://www.opennetworking.org/sdn-resources/sdn-definition*
[4] *https://openflow.stanford.edu/display/ONL/POX+Wiki*
[5] *https://github.com/noxrepo/pox*
[6] *https://www.opennetworking.org/images/stories/downloads/sdn-resources/white-papers/wp-sdn-newnorm.pdf*
[7] *http://mininet.org/sample-workflow/*



**By: Jitendra Bhatia, Shreyans Doshi and Girish Ramnani**

Bhatia works as a senior assistant professor with the CSE department of the Institute of Technology, Nirma University for the past nine years. You can contact him at *jitendra.bhatia@nirmauni.ac.in*

Doshi is a B. Tech final year student in the CSE department at the Institute of Technology, Nirma University. He can be contacted at *Shreyans.doshi94@gmail.com*

Ramnani is a B. Tech pre-final year student in the CSE department at the Institute of Technology, Nirma University. You can contact him at *girishramnani95@gmail.com*